

TÀI LIỆU HƯỚNG DẪN PHP

Chương 1: Cú Pháp Cơ Bản và Quản Lý Biến Số



Tác giả: Chuyên gia Phát triển Web

Ngôn ngữ: PHP 8.x

Định dạng: Tài liệu Kỹ thuật Tài bản Tài liệu

Năm xuất bản: 2026

Mở Đầu Về Ngôn Ngữ PHP

PHP (Hypertext Preprocessor) là một ngôn ngữ lập trình kịch bản mã nguồn mở, được sử dụng rộng rãi và đặc biệt phù hợp cho việc phát triển các ứng dụng web động chạy phía máy chủ (server-side). Để làm chủ PHP, bước đầu tiên và quan trọng nhất là hiểu rõ cách thức hoạt động của cú pháp và cách trình biên dịch xử lý mã lệnh.

1. Thẻ PHP (PHP Tags)

Mã PHP có thể được nhúng trực tiếp vào trong các tài liệu HTML mã nguồn hỗn hợp. Để trình thông dịch PHP nhận biết và xử lý các đoạn mã này, mã lệnh bắt buộc phải được đặt trong các thẻ mở và đóng tiêu chuẩn:

```
<?php
// Mã nguồn PHP được viết ở đây
echo "Xin chào thế giới!";
?>
```

Nếu một tệp tin chỉ chứa hoàn toàn mã nguồn PHP, quy chuẩn lập trình hiện đại (PSR-12) khuyến nghị bắt buộc **không được sử dụng** thẻ đóng `?>` ở cuối tệp. Điều này giúp ngăn ngừa các khoảng trắng hoặc ký tự xuống dòng vô tình lọt vào luồng đầu ra, có thể gây lỗi tiêu đề HTTP (Headers Already Sent).

2. Lệnh Xuất Dữ Liệu (Echo và Print)

Để xuất dữ liệu ra màn hình trình duyệt, PHP cung cấp hai cấu trúc ngôn ngữ phổ biến là `echo` và `print`. Mặc dù trông có vẻ giống nhau, chúng có những điểm khác biệt kỹ thuật quan trọng:

- `echo` : Có thể nhận nhiều tham số cùng lúc, không trả về giá trị và thực thi nhanh hơn một chút về mặt vi mô.
- `print` : Chỉ nhận duy nhất một tham số, luôn trả về giá trị số nguyên `1`, cho phép sử dụng trong các biểu thức toán học phức tạp.

```
<?php
echo "Chuỗi thứ nhất", " Chuỗi thứ hai"; // Hợp lệ với echo
print "Chỉ xuất một chuỗi"; // Hợp lệ với print
?>
```

Lưu ý về Dấu Chấm Phẩy (;): Mỗi câu lệnh thực thi trong PHP bắt buộc phải được kết thúc bằng một dấu chấm phẩy. Thiếu dấu chấm phẩy sẽ dẫn đến lỗi cú pháp nghiêm trọng (Parse Error) và khiến chương trình ngừng chạy lập tức.

Chú Thích Và Bản Chất Của Biến Trong PHP

3. Chú Thích Trong Mã Nguồn (Comments)

Chú thích là các đoạn văn bản giải thích mã nguồn, hoàn toàn bị trình thông dịch bỏ qua khi chạy chương trình. PHP hỗ trợ cả hai phong cách chú thích dòng đơn và dòng khối:

```
<?php
// Đây là chú thích một dòng kiểu C++
# Đây cũng là chú thích một dòng kiểu Unix/Shell

/*
    Đây là chú thích nhiều dòng (khối)
    Dùng để giải thích các thuật toán phức tạp
*/
?>
```

4. Khái Niệm Biến Số (Variables)

Biến số trong lập trình được hiểu như các thùng chứa lưu trữ thông tin tạm thời trong bộ nhớ RAM. Trong PHP, tất cả các biến đều bắt đầu bằng ký tự đô-la \$, theo sau là tên biến.

Quy tắc đặt tên biến hợp lệ:

- Tên biến bắt buộc phải bắt đầu bằng ký tự \$, tiếp sau đó phải là một chữ cái (a-z, A-Z) hoặc một dấu gạch dưới _ .
- Tên biến không được phép bắt đầu bằng một chữ số.
- Tên biến chỉ được chứa các ký tự chữ cái, số và dấu gạch dưới (A-z, 0-9, và _).
- Tên biến có phân biệt chữ hoa và chữ thường (Case-Sensitive). Ví dụ: \$age và \$AGE là hai biến hoàn toàn khác nhau.

```
<?php
$txt = "Hello World!";
$x = 5;
$y = 10.5;
$_valid_name = "Đúng quy chuẩn";
// $4name = "Sai"; // Gây lỗi vì bắt đầu bằng số
?>
```

Cơ chế định kiểu động (Loosely Typed Language): PHP không yêu cầu lập trình viên phải khai báo kiểu dữ liệu của biến trước khi sử dụng. PHP tự động chuyển đổi và gán kiểu dữ liệu phù hợp dựa trên giá trị mà biến đó đang nắm giữ tại thời điểm thực thi.

Kiểu Dữ Liệu Và Phạm Vi Hoạt Động Của Biến

5. Các Kiểu Dữ Liệu Cơ Bản

Biến trong PHP có thể lưu trữ nhiều loại dữ liệu khác nhau nhằm phục vụ cho các mục đích tính toán riêng biệt. Các kiểu dữ liệu nguyên thủy bao gồm:

Kiểu dữ liệu	Mô tả chi tiết	Ví dụ minh họa
String	Chuỗi ký tự đặt trong dấu nháy đơn hoặc kép.	"Học PHP"
Integer	Số nguyên không có phần thập phân (từ âm đến dương).	2026
Float	Số thực có dấu phẩy động hoặc số thập phân.	9.75
Boolean	Trạng thái luận lý logic, chỉ nhận hai giá trị.	true hoặc false
Array	Mảng danh sách lưu trữ nhiều giá trị trong một biến.	[1, 2, 3]

6. Phạm Vi Của Biến (Variable Scope)

Phạm vi quyết định vị trí mà một biến có thể được truy cập hoặc sử dụng trong mã nguồn. PHP có ba phạm vi biến cốt lõi bao gồm: local (cục bộ), global (toàn cục), và static (tĩnh).

Biến Toàn Cục (Global) và Cục Bộ (Local)

Một biến được khai báo bên ngoài tất cả các hàm sẽ có phạm vi **Global** và chỉ có thể truy cập được ở môi trường bên ngoài hàm đó. Ngược lại, một biến được khai báo bên trong một hàm sẽ có phạm vi **Local** và chỉ có thể sử dụng nội bộ bên trong hàm đó.

```
<?php
$x = 5; // Phạm vi Global

def myTest() {
    // echo $x; // LỖI! Không thể truy cập trực tiếp biến global ở đây
    $y = 10; // Phạm vi Local
    echo $y; // Hợp lệ
}
?>
```

Kỹ Thuật Nâng Cao Về Biến Số

7. Từ Khóa global

Để có thể truy cập và chỉnh sửa một biến toàn cục ngay từ bên trong không gian cục bộ của một hàm, PHP cung cấp từ khóa `global` đứng trước tên biến khi gọi.

```
<?php
$x = 5;
$y = 10;

function customAddition() {
    global $x, $y;
    $y = $x + $y;
}

customAddition();
echo $y; // Đầu ra sẽ là 15
?>
```

Ngoài ra, PHP lưu trữ tất cả các biến toàn cục vào một mảng liên hợp hệ thống siêu toàn cục gọi là **`$GLOBALS[index]`**. Chỉ mục `index` chính là tên của biến số cần gọi. Mảng này cũng có thể được truy cập trực tiếp từ bên trong các hàm mà không cần từ khóa `global`.

8. Biến Tĩnh (Static Variable)

Thông thường, khi một hàm hoàn thành việc thực thi, tất cả các biến cục bộ được tạo ra trong hàm đó sẽ bị giải phóng khỏi bộ nhớ máy chủ. Tuy nhiên, nếu bạn muốn giữ lại giá trị của biến cục bộ cho những lần gọi hàm tiếp theo, hãy sử dụng từ khóa `static` khi khai báo:


```
<?php
function keepCount() {
    static $counter = 0;
    $counter++;
    echo $counter . " ";
}

keepCount(); // Xuất ra: 1
keepCount(); // Xuất ra: 2
keepCount(); // Xuất ra: 3
?>
```

Biến tĩnh `$counter` vẫn là biến cục bộ đối với hàm, nghĩa là không thể truy cập từ bên ngoài, nhưng giá trị của nó được bảo toàn qua các chu kỳ chạy hàm khác nhau.

9. Kết Luận Hệ Thống

Việc hiểu sâu sắc về cú pháp, quy tắc đặt tên, các kiểu dữ liệu và mô hình phạm vi biến là nền tảng cốt lõi giúp bạn xây dựng ứng dụng PHP an toàn, tối ưu hiệu năng và dễ bảo trì trong tương lai.